SDG BLOCKCHAIN ACCELERATOR

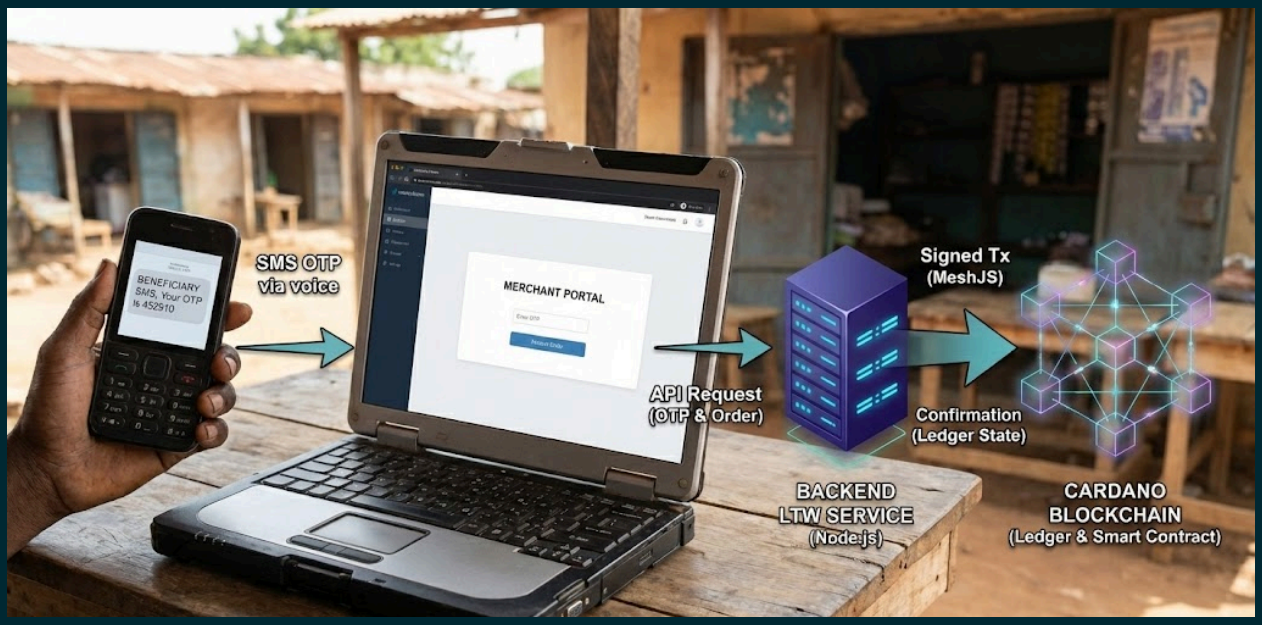
Technical Architecture Document

1. Project Information

- **Project Name:** SHELTER
- **Challenge & UNDP Office:** MALAWI UNDP
- **Document Version:** 1.0

2. Overview

The Low-Tech Wallet is a blockchain-based digital payment solution designed to bring financial inclusion and digital access to the most underserved populations — particularly those living in remote or fragile settings without smartphones, internet connectivity, or access to traditional banking.
Developed in collaboration with UNICEF and local partners such as Kotani Pay and Advantage Africa, the solution enables secure and instant transactions via SMS, allowing users to send, receive, and spend funds using only a basic mobile phone. It bridges humanitarian aid distribution, remittances, and financial services in regions where digital infrastructure is limited.



Alignment with SDG 9 — Industry, Innovation and Infrastructure
SDG 9 calls for building resilient infrastructure, promoting inclusive and sustainable industrialization, and fostering innovation.
The Low-Tech Wallet directly contributes by:
Building resilient digital infrastructure: It leverages blockchain and account abstraction to provide a reliable and transparent financial system that operates even without internet connectivity.
Fostering inclusive innovation: It demonstrates how deep-tech solutions can be adapted to low-resource contexts, lowering the technological barriers to participation in the digital economy.
Enabling local ecosystems: Through integrations with telecom operators and mobile money providers, it strengthens regional payment networks and supports interoperability between humanitarian, public, and private sectors.

Open-source and replicable design: As a Digital Public Good (DPG), its architecture is designed to be reused and localized, creating scalable infrastructure for governments, NGOs, and fintechs.

Alignment with SDG 10 — Reduced Inequalities
SDG 10 focuses on reducing inequalities of income and opportunity, particularly for marginalized and vulnerable populations.
The Low-Tech Wallet advances this goal by:
Providing access to financial services for the unbanked: It allows individuals without smartphones, internet, or bank accounts to receive and use digital funds securely.
Reducing transaction costs: The system enables micro-transactions at costs as low as $0.01 per transfer, making it viable for small-value humanitarian and social transfers.
Empowering women and rural communities: Pilots in Peru, Ghana, and Kenya have shown significant adoption among women, elderly individuals, and remote communities traditionally excluded from digital finance.
Promoting transparency and accountability: Every transaction is recorded on-chain, reducing leakages and ensuring that aid and payments reach the intended recipients.
Impact and Scalability
The solution has already been successfully piloted with UNICEF and partners in Peru and Kenya, demonstrating 100% transaction success rates even without internet connectivity. It can scale across multiple regions — including Africa, Latin America, and Asia — and be integrated with stablecoins or CBDCs on networks like Cardano, Stellar, or Polygon.
By combining blockchain innovation with accessibility, the Low-Tech Wallet is redefining how digital financial infrastructure can serve as a tool for equity, resilience, and inclusion.

3. System Architecture Diagram

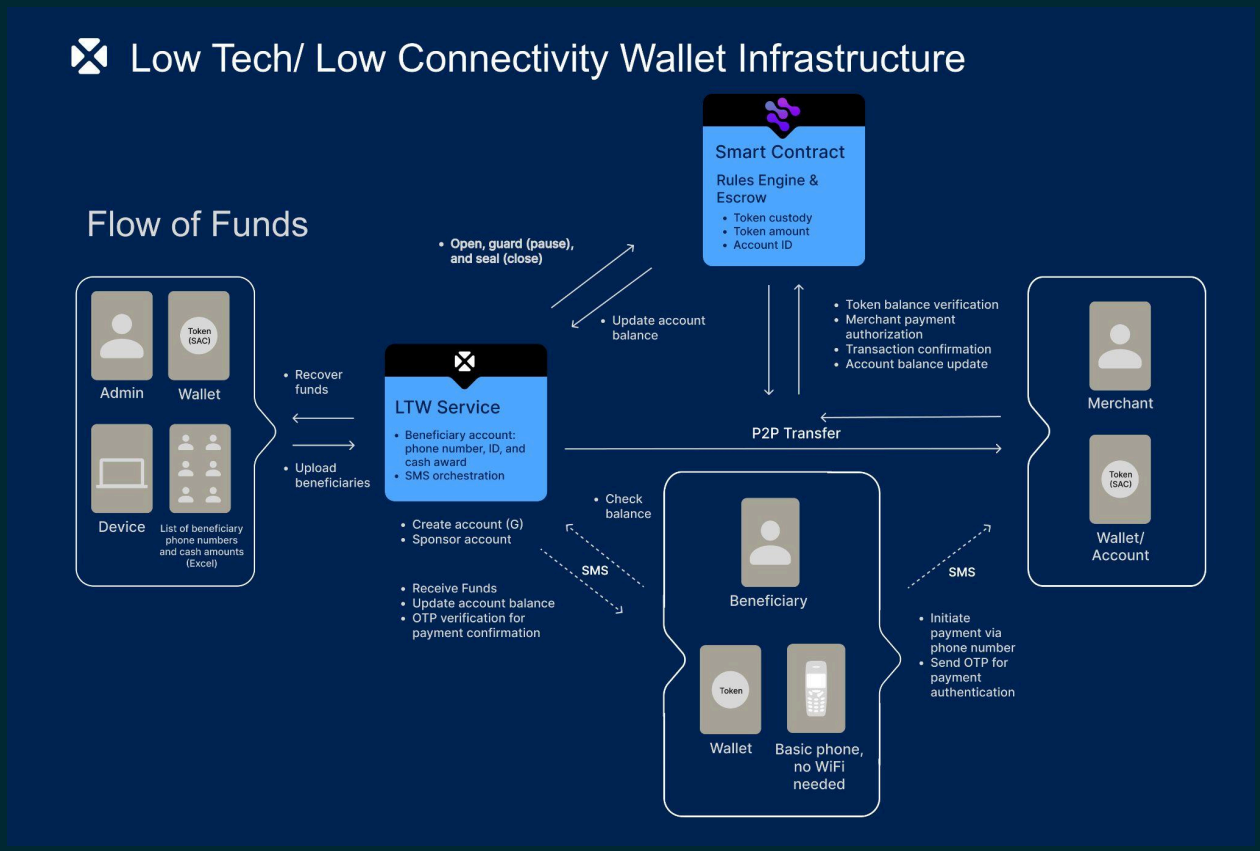
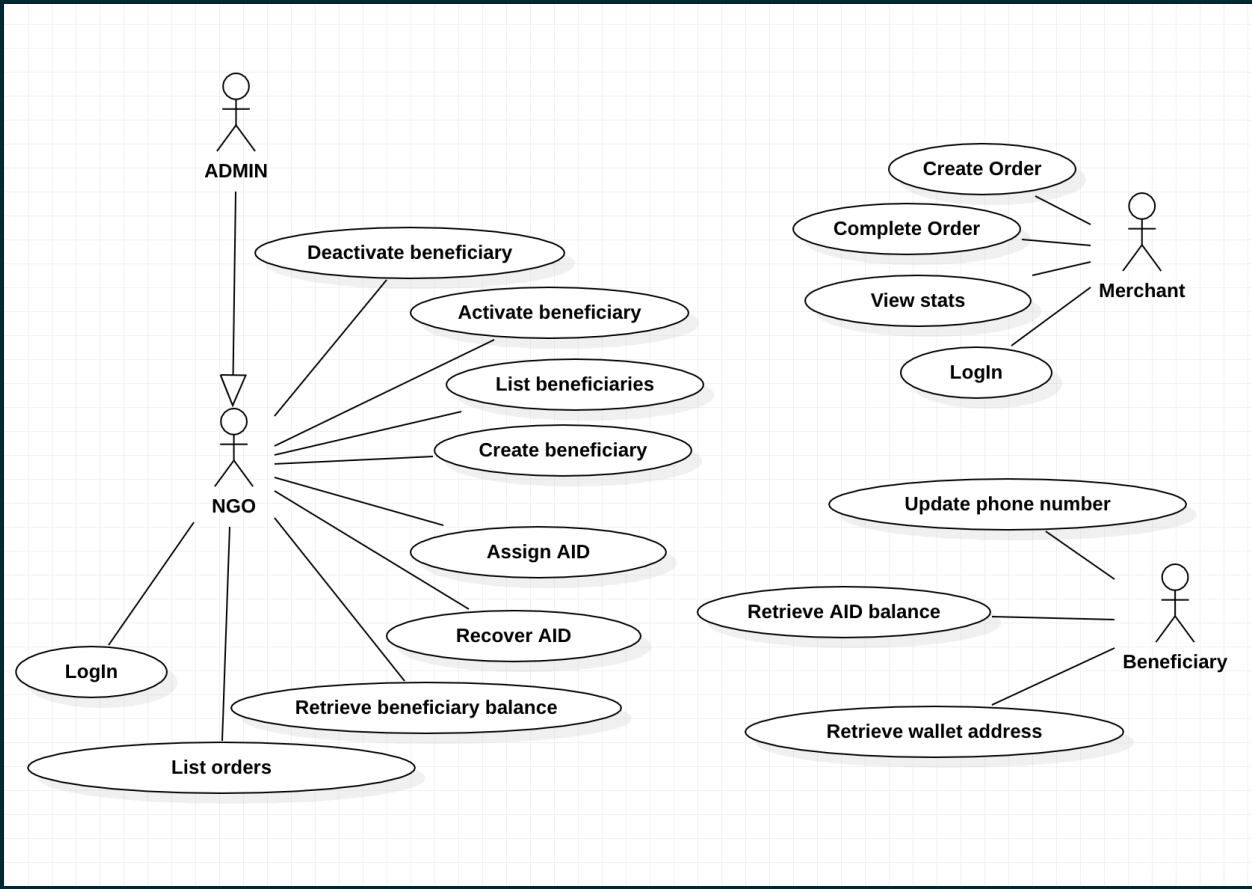


Diagram Infrastructure



UML Use Case Diagram

Assign AID :

```
ADMIN/NGO --(POST)--> LTW Service --> Transaction --> Ledger State
                          |                                ^
                          v                                |
                    (Build Tx)                       (UTxO Locked)
                    (Steward Sign TX)
```

Consume AID :

```
Beneficiary --(OTP)--> Merchant --(POST)--> LTW Service --> Transaction --> Validator Script --> Ledger State
                                                  |                                |                  ^
                                                  v                                v                  |
                                          (Verify OTP)                      (Check Sig)      (1.Funds Released)
                                          (Build Tx)                        (Check Time)     (2.Remaining AID?)
                                          (Beneficiary Sign Tx)                               (2.1.UTxO Locked)
                                          (Steward Sign Tx)
```

Recover AID :

```
ADMIN/NGO --(POST)--> LTW Service --> Transaction --> Validator Script --> Ledger State
                          |                                  |                  ^
                          v                                  v                  |
                     (Build Tx)                          (Check Sig)     (Funds Returned)
                  (Steward Sign Tx)                      (Check Time)
```

4. Blockchain Design

- **Smart Contracts:**
    - Validator script(s) with purpose
        - Spend
    - Datum structure
        - beneficiary: VerificationKeyHash (Hash of the beneficiary).
        - steward: VerificationKeyHash (Hash of the administrator).
        - claim_start: Int (Start POSIX Timestamp).
        - claim_end: Int (Expiration POSIX Timestamp).
    - Redeemer structure
        - `StewardAction`
        - `BeneficiaryAction`

- **UTxO Model Usage:**
    - Each aid package is an individual UTxO containing the ADA and the VoucherDatum.
    - The contract validates the spending of this UTxO based on who signs the transaction (extra_signatories) and the transaction's validity_range.

- **Security Considerations:**
    - **Signature Verification:** Utilizes vodka_extra_signatories to ensure that only the Beneficiary's private key (for consumption) or the Steward's key (for recovery) can authorize the spending.
    - **Time-Lock Logic:** Utilizes vodka_validity_range to prevent premature spending or late claims.
    - **Double-Spend Protection:** Inherent to the Cardano EUTxO model; once the AID is consumed, the UTxO ceases to exist.

5. Data Flow & Transaction Lifecycle

1. Assign AID:

1. The ADMIN/NGO performs a POST /bound-aid request to the LTW Service. The body contains the active beneficiary's phone number, the AID amount in ADA, the start day (e.g., 0 for now), and the expiration day.
2. The Steward submits a transaction to the script address.
3. A UTxO is created with the datum containing the defined hashes and timestamps.
4. State: Funds are locked in the contract.

2. Consume AID:

1. The Merchant creates an order.
2. The Beneficiary receives an SMS with an OTP CODE, which they must dictate to the Merchant.
3. The LTW Service verifies the OTP CODE and order validity. If valid, it constructs the BeneficiaryAction transaction using MeshJS, with the Merchant as the recipient. Any surplus funds are returned to the Beneficiary.
4. The transaction is signed with the Beneficiary's key.
5. The validator checks the Beneficiary's signature and ensures the funds are not expired. If valid, funds are released.
6. State: Funds released to the Merchant/Beneficiary.

3. Recover AID:

1. If current_time > claim_end, the Steward initiates a StewardAction transaction.
2. The validator checks the Steward's signature and verifies that the current time is past the claim_end.
3. State: Funds returned to the Steward.

6. Off-chain Components

- **Merchant Platform (Frontend):** A web application built with Next.js (React) and Tailwind CSS. It utilizes Radix UI for accessible components and Recharts for statistics visualization. It interacts with the blockchain via MeshJS to sign transaction requests and manages form validation using Zod and React Hook Form.
- **Backend Application:** Service developed in Node.js with TypeScript. Acts as a bridge between the telephony network and the blockchain.
- **MeshJS SDK:** Library used for complex transaction construction, UTxO selection, and Datum serialization.

- **Twilio Integration:** Messaging service used to capture user intent (SMS) and convert it into API calls to the backend.
- **Database (FerretDB/PostgreSQL):** Stores the mapping between phone numbers and public keys, as well as the operation history.
- **Blockfrost:** API provider used to query the ledger state and submit signed transactions (Submit Tx).

7. Sandbox/Testnet Results

Transaction Type	Status	CPU	Memory	Notes
Assign AID	Success	N/A	N/A	✅ **Passed OK:** Standard transaction confirmed by Ledger.
Consume AID	Success	3B	7M	✅ **Passed OK:** Validator execution within limits.
Recover AID	Success	3B	7M	✅ **Passed OK:** Validator execution within limits.
Consume AID (without sign beneficiary)	Reject	.	.	✅ **Test Passed:** Security blocked action. Error: *BeneficiaryAction, must be signed with beneficiary key*
Consume AID (before claim start)	Reject	.	.	✅ **Test Passed: Time-lock active. Error: *BeneficiaryAction, must be valid after claim_start***
Consume AID (after claim end)	Reject	.	.	✅ **Test Passed: Expired AID blocked. Error: *BeneficiaryAction, must be valid before claim_end***
Recover AID (without sign steward)	Reject	.	.	✅ **Test Passed:** Security blocked action. Error: *StewardAction, must be signed with steward key*

Recover AID (before claim end)	Reject	.	.	✅ **Test Passed: Active AID protected. Error:** ***StewardAction, must be valid after claim_end***

8. Tools and Environments Used

Blockchain & Smart Contracts

- Cardano Preview Testnet
- Aiken v1.1.17
- Vodka / Cocktail / Mocktail
- Cardano Node & CLI
- Blockfrost

Frontend Ecosystem (Merchant Platform)

- **Framework:** Next.js 16 (React 19)
- **UI & Styling:** Tailwind CSS v4 & Radix UI
- **Blockchain Integration:** Mesh SDK (@meshsdk/core)
- **Validation:** Zod & React Hook Form
- **Testing:** Vitest & Playwright

Backend Ecosystem (LTW Service)

- **Runtime:** Node.js (v20+) & TypeScript
- **Testing:** Jest
- **Blockchain Integration:** Mesh SDK (@meshsdk/core)
- **Integrations:** Twilio SDK & Axios
- **Database:** PostgreSQL & FerretDB
- **Infrastructure:** Docker

9. Remaining Considerations / Next Steps

- **Production Refactoring (Trace Removal)**: Refactor Aiken contract debug messages (trace) into expect statements to reduce script size and execution costs prior to Mainnet deployment.
- **LTW Service Optimization**: Refactor backend components to improve modularity and achieve Code Coverage above 80%.

- **Merchant Platform Refactoring**: Refactor the Merchant interface architecture and introduce automated component testing to ensure UI reliability and seamless interaction with the blockchain.
- **Preprod Stress Testing**: Migrate the testing environment to the Preprod network to simulate high-concurrency scenarios and validate UTxO management scalability.
- **Security Audit**: Conduct an external security audit focusing on the time-lock logic and backend key management.
- **Rahat Integration**: Evaluate the integration of the Rahat Dashboard (https://docs.rahat.io/) to provide NGOs with advanced visualization tools for real-time aid tracking and interoperability with global humanitarian standards.